ESGI
école supérieure de génie informatique

Séance 2 - Collections

Au programme

- Conception du schéma, puis kit de données de votre sujet
- CRUD : les cinq opérations d'une collection
- `PUT` vs `PATCH` : remplacer ou modifier
- Le problème d'une collection qui grossit
- Pagination offset et enveloppe de réponse
- Filtrage, tri stable, recherche
- Aparté : le problème N+1 côté REST

© 2026 Nicolas FAESSEL

CRUD : les opérations d'une collection

Opération	Requête	Réponse en cas de succès
Créer	`POST /ouvrages`	`201 Created` + `Location`
Lister	`GET /ouvrages`	`200 OK` + collection
Lire un élément	`GET /ouvrages/128`	`200 OK` + ressource
Remplacer	`PUT /ouvrages/128`	`200 OK` ou `204 No Content`
Modifier	`PATCH /ouvrages/128`	`200 OK`
Supprimer	`DELETE /ouvrages/128`	`204 No Content` ; `404` si absent

> Cinq verbes, une collection : un client qui connaît ce tableau sait déjà utiliser l'API sans lire de doc.

© 2026 Nicolas FAESSEL

`PUT` vs `PATCH`

`PUT` **remplace** la ressource entière. Les champs absents du corps sont considérés comme effacés.

```
PUT /adherents/42
Content-Type: application/json

{ "nom": "Bob", "email": "bob@ex.fr", "actif": true }
```

`PATCH` **modifie** les champs fournis, laisse les autres tels quels.

```
PATCH /adherents/42
Content-Type: application/json

{ "email": "nouveau@ex.fr" }
```

> Envoyer un corps partiel en `PUT` est un bug fréquent : les champs manquants sont perdus.

© 2026 Nicolas FAESSEL

Une collection qui grossit

`GET /ouvrages` sur un catalogue de 40 000 titres :

- réponse énorme, lente à produire, lente à transférer, lente à parser
- le client n’a besoin que des 20 premiers
- la base souffre à chaque appel

> Une collection ne se renvoie **jamais** en entier. On la découpe en pages, et on donne au client de quoi demander la suivante.

© 2026 Nicolas FAESSEL

Pagination offset

Le client demande une tranche par sa position :

```
GET /ouvrages?page=3&limit=20
```

ou, équivalent, `?offset=40&limit=20`.

- simple à implémenter (`LIMIT ... OFFSET ...` en SQL)
- permet de sauter directement à la page N
- **dérive** si des éléments sont insérés ou supprimés pendant la navigation : on peut sauter ou revoir un élément d'une page à l'autre

La pagination par **curseur**, qui corrige cette dérive, est vue en séance 4.

© 2026 Nicolas FAESSEL

L’enveloppe de réponse

Une collection paginée renvoie les éléments **et** de quoi se repérer :

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "data": [ { "id": 128, "titre": "Dune" }, "..." ],
  "pagination": { "page": 3, "limit": 20, "total": 402 },
  "links": { "next": "/ouvrages?page=4&limit=20", "prev": "/ouvrages?page=2&limit=20" }
}
```

> Choisissez une forme d’enveloppe et tenez-la sur **toutes** vos collections. Le client écrit son code de pagination une seule fois.

© 2026 Nicolas FAESSEL

Filtrage

Les critères de restriction passent en paramètres de requête, combinables :

```
GET /ouvrages?auteur=herbert&disponible=true&annee_min=1960
```

- chaque paramètre est optionnel
- absent = pas de filtre sur ce critère
- un paramètre inconnu : l'ignorer, ou renvoyer `400` - à décider et à documenter

© 2026 Nicolas FAESSEL

Tri, et le piège du tri instable

```
GET /ouvrages?tri=titre&ordre=asc
```

Si plusieurs ouvrages ont le même titre, leur ordre relatif n'est **pas garanti** d'un appel à l'autre. En paginant, un élément peut apparaître page 2 puis page 3.

> **Tri stable** : ajouter toujours une clé de départage unique, par exemple `tri=titre,id`. L'ordre devient déterministe, la pagination cohérente.

© 2026 Nicolas FAESSEL

Recherche simple

Un paramètre `q` pour une recherche plein texte basique :

```
GET /ouvrages?q=épice&limit=10
```

- porte sur un ou plusieurs champs (titre, auteur, résumé) - à documenter
- se combine avec le filtrage et la pagination
- la recherche avancée (pertinence, facettes) est hors périmètre de ce module

© 2026 Nicolas FAESSEL

Le problème N+1

Lister 20 ouvrages, puis pour chacun charger ses exemplaires : 21 allers-retours pour une page.

```
SELECT * FROM ouvrages ORDER BY titre, id LIMIT 20       -- 1 fois
SELECT * FROM exemplaires WHERE ouvrage_id = ?           -- 20 fois
```

Remède : 2 requêtes, quelle que soit la taille de la page.

```
SELECT * FROM exemplaires WHERE ouvrage_id IN (128, 131, ...)   -- 1 fois
```

- pas de jointure naïve : `LIMIT 20` compterait 20 lignes jointes, pas 20 ouvrages
- côté contrat : lien vers la relation, ou relation embarquée à la demande (`?include=exemplaires`)

> On le retrouve en séance 7 : en GraphQL, chaque resolver ne voit qu'un parent, le N+1 y apparaît naturellement.

© 2026 Nicolas FAESSEL

TP

© 2026 Nicolas FAESSEL

TP - CRUD et collection navigable

Objectif : implémenter le modèle de ressources de votre sujet et rendre une collection paginable, filtrable et triable.

- Handout : `supports/tp/session-02.md`
- Kit de données, remis après 10 min de conception : `supports/kits/<sujet>/`
- **Livrable attendu** :
  - esquisse du modèle de données dans `docs/api-design.md`
  - API qui démarre et répond ; CRUD complet sur au moins 2 ressources
  - une collection paginée (offset) + filtrable + triable, avec enveloppe de réponse
  - début de la collection de requêtes versionnée (`requests/`)
  - stratégie de pagination notée en une phrase dans le `README`

© 2026 Nicolas FAESSEL